

Experiment 1

Title

Implementation of Finite Automata for Regular Languages

Aim

To understand the concept of Finite Automata and implement Deterministic Finite Automata (DFA) in C for recognizing Regular Languages.

Software Requirements

- GCC Compiler / Turbo C++ / CodeBlocks / Visual Studio Code
- Operating System: Windows / Linux

Prerequisites

Students should know:

- Tokens
- Regular Expressions
- DFA
- States
- Transition Diagram

Theory

A **Finite Automaton (FA)** is a mathematical model used to recognize patterns in strings.

In Compiler Design, the **Lexical Analyzer** uses Finite Automata to recognize tokens such as:

- Identifiers
- Keywords
- Numbers
- Operators
- Delimiters

Instead of checking every possible string manually, the compiler moves through a sequence of **states**.

After reading the complete input:

- If the automaton reaches an **accepting state**, the string is accepted.
- Otherwise, it is rejected.

Thus, Finite Automata provide an efficient mechanism for token recognition during lexical analysis.

Real Compiler Connection

Consider the C statement:

```
int marks = 100;
```

The lexical analyzer reads:

$i \rightarrow n \rightarrow t$

and recognizes it as a:

KEYWORD

Then it reads:

$m \rightarrow a \rightarrow r \rightarrow k \rightarrow s$

and recognizes it as an:

IDENTIFIER

This recognition process is performed using **Finite Automata**.

Inside the Compiler

Source Program



Characters



Finite Automata



Recognized Tokens



Parser

Conclusion

In this experiment, we studied the implementation of a **Deterministic Finite Automaton (DFA)** in C.

We learned how:

- States are represented using variables.
- Transitions are implemented using conditional statements.
- Input is processed one character at a time.
- Acceptance and rejection are determined by the final state.